

SCJP 1.6 (CX-310-065 , CX-310-066)

Subject: Mock Questions Set-2

Total Questions : 60

Prepared by : <http://www.javacertifications.net>

SCJP 6.0: Full Length Mock Exam Set-2(60 Questions)

Questions Question - 1

If you run the code below, what gets printed out?

```
public class Test {  
  
    public static void main(String argv[]){  
        String s=new String("Bicycle");  
        int iBegin=1;  
        char iEnd='a';  
        System.out.println(s.substring(iBegin,iEnd));  
    }  
}
```

- 1.Bic
- 2.icy
- 3.ic
- 4.error: no method matching substring(int,char)

Explanation :

C is the correct answer.

if you declare char iEnd='a'; then it converts to 97 (ASCII value of 'a' is 97).

"Bicycle".substring(1,97) throws java.lang.StringIndexOutOfBoundsException:

String index out of range: 97

Question - 2

Which of the following can be applied to constructors ?

- 1.final
- 2.static
- 3.synchronized
- 4.None of the above

Explanation :

D is the correct answer.

A constructor cannot be abstract, final, native, static or synchronized

Question - 3

Which of the following statements are correct regarding the `RandomAccessFile` class?

- 1) An `IOException` is thrown if the specified file doesn't exist when created using the "r" mode.
- 2) This class has a method which allows a file to be deleted from the hard disk.
- 3) It is possible to use this class in conjunction with the `DataInputStream` class.
- 4) When used with the "rw" mode, the specified file is created on a disk drive, if it doesn't already exist.
- 5) There are methods to read and write primitives (eg, `readInt()`, `writeInt()`, etc).

- 1.1 , 4 and 5
- 2.1 and 2
- 3.1 , 3 and 5
4. None of the above

Explanation :

A is the correct answer.

Answer 1 is correct, because in read mode, the file must exist if any other operations are to be performed on it. The `RandomAccessFile` is not compatible with the `Stream` or `reader/writer` classes, so Answer 3 is incorrect.

Question - 4

Which of the following keywords can be applied to a method?

- 1) `transient`
- 2) `volatile`
- 3) `private`
- 4) `protected`
- 5) `final`

- 1.1 , 2 and 5
- 2.1, 2 and 3
- 3.3 , 4 and 5
4. None of the above

Explanation :

C is the correct answer.

"transient" is a keyword used for preventing variables in an object from being serialised. It cannot be applied to methods. "volatile" is used to inform the compiler that a variable may be changed asynchronously with regard to its current definition (eg, from a native method outside the JVM). The compiler will not perform any optimisations on a variable declared as volatile. Methods can be : private (not accessible outside the containing class) protected (only accessible to the containing class and subclasses) final (not allowed to be overridden).

Question - 5

Consider the following piece of code and select the correct statement(s):

```
public class Test{
    final int x = 0;
    Test(){
        x = 1;
    }
    final int aMethod(){
        return x;
    }
}
```

- 1.The code fails to compile. The compiler complains because there is a final method ("aMethod") in a non-final class.
- 2.The code compiles correctly. On execution, an exception is thrown when the Test() constructor is executed.
- 3.The code fails to compile because an attempt is made to alter the value of a final variable.
- 4.The code fails to compile because only methods can be declared as final (and therefore "final int x = 0" is not valid).

Explanation :

C is the correct answer.

Answer 1 is incorrect, because a class can have final methods and variables without the need for the entire class to be final. Answer 2 is incorrect, because the type of error in this code is detectable at compile time.

Question - 6

What is the valid declaration for the finalize() method.

- 1.protected void finalize() throws Throwable
- 2.final finalize()

3. public final finalize()
4. private boolean finalize()

Explanation :

A is the correct answer.

protected void finalize () throws Throwable. The finalize() method is declared in the java.lang.Object class. Thus when you write a finalize() method for your class you are overriding the one in your superclass.

Question - 7

Consider the following piece of code.

```
class Test{
public static void main(String [] args){
System.out.println(args[3]);
}
}
```

When the following is typed at the command line, what is displayed:

```
java Test hello Justice For All
```

1. All
2. For
3. hello
4. Nothing.

Explanation :

A is the correct answer.

The command line args are passed in, indexed from zero. The args excludes the program name ("Test").

Question - 8

Which of the following are valid ways to define an octal literal of value 17 (octal).

1. private final int theNumber = 017;
2. public int theNumber = 017;
3. public int THE_NUMBER = 017;
4. All of the above

Explanation :

D is the correct answer.

The answers contain various spurious keywords in an attempt to confuse Answer 1 is incorrect, because it defines a hexadecimal number (0x17). Answer 4 is incorrect because there is no such keyword as "octal".

Question - 9

Which of the following are methods within the String class.

- 1.substring()
- 2.startsWith()
- 3.toString()
- 4.All of the above

Explanation :

D is the correct answer.

Refer to the JDK API documentation.

Question - 10

Consider the following code segment and select the correct statement(s) :

```
1.  class FinalTest{
2.      final int q;
3.      final int w = 0;
4.
5.      FinalTest(){
6.          q = 1;
7.      }
8.
9.      FinalTest(int x){
10.         q = x;
11.     }
12. }
```

- 1.The code fails to compile because a class cannot have more than 1 constructor.
- 2.The code fails to compile because the class FinalTest has no constructors.
- 3.The code fails to compile because the an attempt is made to initialise a final variable at lines 6 and 10.
- 4.The code compiles correctly without any warnings or errors.

Explanation :

D is the correct answer.

The code segment is valid. It is allowable to initialise a final variable BEFORE IT IS USED, providing it was not assigned a value statically (ie, at declaration).

Question - 11

When is the text "Hello" displayed?

```
public class StaticTest {
static {
```

```

System.out.println("Hello");
}

public void print() {
System.out.println("Print");
}

public static void main(String args []) {
StaticTest st1 = new StaticTest();
st1.print();
StaticTest st2 = new StaticTest();
st2.print();
}
}

```

- 1.Never.
- 2.Each time a new object of type StaticTest is created.
- 3.Once when the class is loaded into the JVM.
- 4.Only when the main() method is executed.

Explanation :

C is the correct answer.

The code in this segment is what is known as a static initialiser. Static initialiser code is executed once when the class is loaded into the JVM. Multiple static initialiser blocks in a single class are executed in the order they are defined. Static initialisers are not related to the main() method. Therefore, Answers 1, 2 and 4 are incorrect.

Question - 12

Which of the following statements is correct with regard to threads.

- 1."Runnable" is a keyword within the Java language, used to identify classes which can exist as separate threads.
- 2."Runnable" is an interface, which classes can implement when they wish to execute as a separate thread.
- 3."Runnable" is a class, which classes can extend when they wish to execute as a separate thread.
- 4.None of the above.

Explanation :

B is the correct answer.

Runnable is an abstract interface.

Question - 13

In a thread, the wait() method must be called inside which of the following:

- 1.A while() loop
- 2.The run() method
- 3.synchronised code
- 4.The constructor

Explanation :

C is the correct answer.

The wait() method can only be called inside synchronised code. Refer to the Java Language Specification.

Question - 14

What is displayed when the following piece of code is compiled and executed:

```
class Test{
public static void main(String [] args){
Base b = new Subclass();
System.out.println(b.x);
System.out.println(b.method());
}
}
class Base{
int x = 2;
int method(){
return x;
}
}
class Subclass extends Base{
int x = 3;
int method(){
return x;
}
}
```

- 1.Nothing. The code fails to compile because the object b is not created in a valid way.
- 2.2 3
- 3.2 2
- 4.3 3

Explanation :

B is the correct answer.

The rule for accessing methods and variables in a class hierarchy is as follows: The object reference is used when accessing variables (so the value of b.x is therefore 2). the underlying object is used when accessing methods (so the method called is actually the method defined in Subclass, and in Subclass, x =3). Therefore answer 2 is correct.

Question - 15

True or False.

Anonymous classes cannot have constructors.

- 1.True
- 2.False
- 3.None of the above
- 4.None of the above

Explanation :

A is the correct answer.

Anonymous classes are classes which do not have names. Therefore, constructors are meaningless.

Question - 16

What is displayed when the following piece of code is executed:

```
class Test extends Thread{
public void run() {
System.out.println("1");
yield();
System.out.println("2");
suspend();
System.out.println("3");
resume();
System.out.println("4");
}
public static void main(String []args) {
Test t = new Test();
t.start();
}
}
```

- 1.1 2 3 4
- 2.1 2 3
- 3.1 2
- 4.Nothing. This is not a valid way to create and start a thread.

Explanation :

C is the correct answer.

The code will run, but the thread suspends itself after displaying "2". Threads cannot un-suspend themselves (since they ARE suspended and therefore not running!).

Question - 17

Which of the following interfaces can be used to manage a collection of elements, with no duplication.

- 1.List
- 2.Vector
- 3.Set
- 4.Map

Explanation :

C is the correct answer.

Lists and Vectors allow duplication (so answers 1 and 2 are incorrect).

Question - 18

Which of the following are valid ways to create a Map collection.

- 1.Map m = new Map();
- 2.Map m = new Map(init capacity, increment capacity);
- 3.Map m = new Map(new Collection());
- 4.Map is an interface, and cannot be instantiated.

Explanation :

D is the correct answer.

Map is an interface, and cannot be instantiated. You can create instance of HashMap.

Question - 19

What is the result of compiling and executing the following code.

```
public class Test extends Thread {
    public void run() {
        System.out.println("start");
        yield();
        System.out.println("end");
    }
    public static void main(String args []) {
        (
            new Test()).start();
    }
}
```

- 1.The code fails to compile in the main() method.
- 2.The code fails to compile in the run() method.
- 3.Only the text "start" will be displayed.
- 4.The text "start" followed by "end" will be displayed.

Explanation :

D is the correct answer.

yield() Causes the currently executing thread object to temporarily pause and allow other threads to execute. The yield() method simply returns control back to the JVM, and unless there is a higher priority thread ready to run, this thread will continue or resume execution. Answer 1 is incorrect. The main() method is defined correctly, and the line (new Test()).start(); is a valid way to create and start a Test thread (since the class Test is a subclass of Thread). Answer 2 is incorrect, since the run() method is declared correctly. The yield() method simply returns control back to the JVM, and unless there is a higher priority thread ready to run, this thread will continue or resume execution. Answer 3 is incorrect, since the yield() only has a temporary effect on execution.

Question - 20

What is the output for the below code ?

```
public class Test{
    int x = 0;

    public Test(int inVal) throws Exception{
        if (inVal != this.x){
            System.out.println("same data");
            throw new Exception("Invalid");
        }
    }

    1.    public static void main(String args[]){
    2.        Test t = new Test(4);
    3.    }
}
```

- 1.Invalid
- 2.same data.
- 3.Compile successfully
- 4.The code fails to compile at line 2. The compiler complains that there is an uncaught exception.

Explanation :

D is the correct answer.

Answer 4 is correct. In order to make the code compile, you need to wrap the creation of the object in a try...catch block.

Question - 21

Suppose a MyException should be thrown if Condition() is true, which statements do you have to insert ?

```
1: public aMethod {
2:
3:     if (Condition) {
4:
5:     }
```

```
6:
7: }
```

- 1.throw new Exception() at line 4
- 2.throw new MyException() at line 4
- 3.throws MyException at line 1
- 4.throws IOException at line 1

Explanation :

B and C is the correct answer.

You need to do 2 things...declare the method to throw an exception (answer 3) and then throw the exception (answer 2).

Question - 22

Consider the following class:

```
public class Test{
public int secret;
}
```

Which of the following is the correct way to make the variable 'secret' read-only.

- 1.Declare 'secret' to be private.
- 2.Declare the class Test to be private.
- 3.Declare 'secret' to be private and write a method getSecret() which returns the value of 'secret'.
- 4.Declare 'secret' to be transient.

Explanation :

C is the correct answer.

Answer 1 is incorrect, because doing this will result in secret being inaccessible from outside the class. Answer 2 is incorrect, because there is no such thing as a private class (unless it is an inner class). Answers 4 is misleading, in that neither if these affect the scope or accessibility of the variable.

Question - 23

What outputs are possible from invoking Math.random() .

- 1.-0.12 0.56E3
- 2.0.12 1.1E1
- 3.-23.45 0.0
- 4.0.356 0.03

Explanation :

D is the correct answer.

The output for the random() method is a random number greater than or equal to 0.0 and less than 1.0

Question - 24

What is the result of the following code :

```
public class SuperEx {
    String r;
    String s;
    public SuperEx(String a,String b) {
        r = a;
        s = b;
    }
    public void aMethod() {
        System.out.println("r :" + r);
    }
}

public class NewSuper extends SuperEx {
    public NewSuper(String a,String b) {
        super(a,b);
    }
    public static void main(String args []) {
        SuperEx a = new SuperEx("Hi", "Tom");
        SuperEx b = new NewSuper("Hi", "Bart");

        a.aMethod();
        b.aMethod();
    }
    public void aMethod() {
        System.out.println("r :" + r + " s:" + s);
    }
}
```

- 1.The following is displayed: r:Hi s:Hi
- 2.Compiler error at the line "SuperEx b = new NewSuper("Hi","Bart");"
- 3.The following is displayed: r:Hi r:Hi s:Bart
- 4.The following is displayed r:Hi s:Tom r:Hi s:Bart

Explanation :

C is the correct answer.

None.

Question - 25

Consider the following piece of code and select the correct statements.

1. `Object o = new String("abcd");`
2. `String s = o;`
3. `System.out.println(s);`
4. `System.out.println(o);`

- 1.The following is displayed: abcd abcd
- 2.The code fails to compile at line 1.
- 3.The code fails to compile at line 2
- 4.The code fails to compile at line 4.

Explanation :

C is the correct answer.

The code fails to compile because line 2 is attempting to assign objects "down" the hierarchy.

Question - 26

Which of the following keywords can be applied to the variables or methods of an interface.

- 1.static
- 2.private
- 3.public
- 4.1 and 3

Explanation :

D is the correct answer.

Refer to the Java language specification.

Question - 27

Consider the following.

```
String s1 = "abc";  
String s2 = "def";  
String s3 = new String(s2);  
s2 = s1;
```

What is the value of s3 after the final line of code is executed?

- 1."abc"
- 2."def"
- 3.null
- 4.An exception is thrown when "s2 = s1" is executed.

Explanation :

B is the correct answer.

Once s3 is created, it is unaffected by anything happening to s2.

Question - 28

What is displayed when the following code is executed:

```
String s = "abcdef";  
System.out.println(s.charAt(4));
```

- 1.d
- 2.e
- 3.Nothing. An ArrayIndexOutOfBoundsException is thrown
- 4.The code does not compile. charAt() is not a valid method of the String class.

Explanation :

B is the correct answer.

Indices are zero based, so the character at index 4 is "e".

Question - 29

You get this description of a class :

Employee is a person. For every employee, we keep a vector with the working hours, an integer for the salary and a variable that can be true or false whether or not the employee has a company car. Indicate which of the following would be used to define the class members.

- 1.Vector
- 2.Employee
- 3.Object
- 4.boolean

Explanation :

A is the correct answer.

A Vector is used for storing the working hours (see the question text!). A boolean is used for the company car flag. An int is used for the salary.

Question - 30

True or False.

Abstract methods can be declared as static.

- 1.True
- 2.False
- 3.Can't say
- 4.None of the above

Explanation :

B is the correct answer.

Abstract methods cannot be declared as static, final or private. Refer to the Java Language Specification (section 8.4.3.1, version 1.0).

Question - 31

```
Consider the following:  
class A extends Integer{  
int x = 0;  
}  
Select all valid statements.
```

- 1.The code will compile correctly.
- 2.The code will not compile because Integer is final and cannot be subclassed.
- 3.The code will not compile because class A has no methods or constructor.
- 4.The code will compile correctly, but will throw an ArithmeticException at runtime.

Explanation :

B is the correct answer.

Answer 3 is incorrect. A class does not need a method or constructors to exist.
Answer 4 is a meaningless comment.

Question - 32

Which of the following classes override the equals() method.

- 1.String
- 2.Integer
- 3.Double
- 4.All of the above

Explanation :

D is the correct answer.

Refer to the JDK documentation.

Question - 33

You want to extend the functionality of the String class, and decide that the best approach is to subclass String.
Which of the following statements are correct.

- 1.The approach fails, because the String class is declared as final and cannot be

subclassed.

- 2.The approach succeeds, and it is therefore possible to override the standard functionality provided by String.
- 3.The approach fails, because the String class is declared as abstract, and therefore it cannot be subclassed.
- 4.None of the above

Explanation :

A is the correct answer.

Answer 1 is correct. String is declared as final, so it cannot be extended. Answer 2 is incorrect (see Answer 1 explanation above). Answer 3 is incorrect in several areas. Firstly, the String class is not abstract. Secondly, even if it was abstract, it can still be subclassed (abstract means that an instance cannot be created - it doesn't affect subclassing).

Question - 34

Which statement is true about String ?

- 1.String objects are immutable, meaning that once created they cannot be altered
- 2.String objects are mutable, meaning that once created they can be altered
- 3.String objects are mutable, meaning that once created they cannot be altered
- 4.None of the above

Explanation :

A is the correct answer.

String objects are immutable, meaning that once created they cannot be altered.

Question - 35

Fill in all the gaps using the available fragments:

```
public final class String
    implements , Comparable<String>, CharSequence
}
```

Use the following fragments zero or many times

Serializable

Comparator

Question - 36

Fill in all the gaps using the available fragments:

```
public class String
    implements , Comparable<String>, CharSequence
```


}

Use the following fragments zero or many times

Serializable

Comparator

final

static

Question - 37

Fill in all the gaps using the available fragments:

```
public class StringBuffer
    extends AbstractStringBuilder
    implements , CharSequence
{
}
```

Use the following fragments zero or many times

Serializable

Comparator

final

static

Question - 38

What is the output?

```
public class Test {

    public static void main(String[] args) {

        StringBuffer sb = new StringBuffer("ssss");
        StringBuffer sb_2 = new StringBuffer("ssss");
        System.out.println("sb equals sb_2 : " + sb.equals(sb_2));

    }

}
```

- 1.sb equals sb_2 : false
- 2.sb equals sb_2 : true
- 3.Compile with error
- 4.None of the above

Explanation :

A is the correct answer.

StringBuffer class DOES NOT override the equals() method. Therefore, it uses Object class' equals(), which only checks for equality of the object references

Question - 39

What is the output?

```
public class Test {  
  
    public static void main(String[] args) {  
  
        StringBuffer sb = new StringBuffer("ssss");  
        String st = new String("ssss");  
        System.out.println("st equals sb : " + st.equals(sb));  
  
    }  
  
}
```

- 1.st equals sb : false
- 2.st equals sb : true
- 3.Compile Error
- 4.Runtime Exception

Explanation :

A is the correct answer.

String's equals() method checks if the argument is of type string, if not it returns false

Question - 40

What is the output of the below code ?

```
class c1  
{  
    public static void main(String a[])  
    {  
        c1 ob1=new c1();  
        Object ob2=ob1;  
        System.out.println(ob2 instanceof Object);  
        System.out.println(ob2 instanceof c1);  
    }  
}
```

- 1.Prints true,false
- 2.Print false,true
- 3.Prints true,true
- 4.compile time error

Explanation :

C is the correct answer.

instanceof operator checks for instance related to class or not.

Question - 41

```
public class A{
    public void test1(){

        System.out.println("test1 A");
    }
}

public class B extends A{
    public void test1(){

        System.out.println("test1 B");
    }
}

public class Test{
    public static void main(String[] args){
        A b = new B();
        b.test1();

    }
}
```

What is the output?

- 1.test1 B
- 2.test1 A
- 3.Not compile because test1() method in class A is not visible
- 4.None of the above

Explanation :

A is the correct answer.

Superclass reference of subclass point to subclass method and superclass variables.

Question - 42

What is the output?

```
import java.util.Iterator;
import java.util.TreeSet;

public class Test {
```

```

public static void main(String... args) {

    TreeSet s1 = new TreeSet();
    s1.add("one");
    s1.add("two");
    s1.add("three");
    s1.add("one");

    Iterator it = s1.iterator();
    while (it.hasNext() ) {
        System.out.print( it.next() + " " );
    }

}
}

```

- 1.one three two
- 2.Runtime Exception
- 3.one three two one
- 4.one two three

Explanation :

A is the correct answer.

V TreeSet assures no duplicate entries.it will return elements in natural order, which, for Strings means alphabetical.

Question - 43

Which collection class grows or shrinks its size and provides indexed access to its elements, but methods are not synchronized?

- 1.java.util.ArrayList
- 2.java.util.List
- 3.java.util.HashSet
- 4.java.util.Vector

Explanation :

A is the correct answer.

☐ ArrayList provides an index to its elements and methods are not synchronized.

Question - 44

Which most closely matches a description of a Java Map?

- 1.A vector of arrays for a 2D geographic representation
- 2.A class for containing unique array elements
- 3.A class for containing unique vector elements
- 4.An interface that ensures that implementing classes cannot contain duplicate keys

Explanation :

D is the correct answer.

Map does not allow duplicate key. It override the the value.

Question - 45

Is Map interface extends Collection interface ?

- 1.true
- 2.false
- 3.can't say
- 4.None of the above

Explanation :

B is the correct answer.

```
public interface Map { }
```

Question - 46

List<String> list = new ArrayList<String>
In the above list, is it only can contain String objects ?

- 1.yes, it only can contain String objects
- 2.No, it can contain any objects
- 3.it can contain only String object and Colletion Object
- 4.None of the above

Explanation :

A is the correct answer.

List list = new ArrayList In the above list, it only can contain String objects

Question - 47

Which statement is true ?

- 1.TreeSet is slower than HashSet.
- 2.HashSet is slower than TreeSet .
- 3.Both has the same performance.
- 4.None of the above

Explanation :

A is the correct answer.

TreeSet, which stores its elements in a red-black tree, orders its elements based on their values; it is substantially slower than HashSet

Question - 48

What is the output of bellow code ?

```
public class Bean{
    private String str;

    Bean(String str ){
        this.str = str;
    }

    public String getStr() {
        return str;
    }

    public boolean equals(Object o){
        if (!(o instanceof Bean)) {
            return false;
        }

        return ((Bean) o).getStr().equals(str);
    }

    public int hashCode() {
        return 12345;
    }

    public String toString() {
        return str;
    }
}

import java.util.HashSet;

public class Test {

    public static void main(String ... sss) {
        HashSet myMap = new HashSet();
        String s1 = new String("das");
        String s2 = new String("das");
        Bean s3 = new Bean("abcdef");
        Bean s4 = new Bean("abcdef");

        myMap.add(s1);
        myMap.add(s2);
        myMap.add(s3);
        myMap.add(s4);

        System.out.println(myMap);
    }
}
```

- 1.das abcdef
- 2.das abcdef das abcdef
- 3.das das abcdef abcdef

4. None of the above

Explanation :

A is the correct answer.

implemented 'equals' and 'hashCode' methods to get unique result in Set.

Question - 49

NullPointerException thrown when

1. Accessing or modifying the field of a null object
2. Taking the length of null as if it were an array
3. Calling the instance method of a null object
4. None of the above

Explanation :

A, B and C is the correct answer.

java.lang.Exception extends java.lang.Throwable.

Question - 50

Is this statement is true?

"java.lang.Throwable class is superclass of all Exceptions"

1. true
2. false
3. can't say
4. None of the above

Explanation :

A is the correct answer.

java.lang.Exception extends java.lang.Throwable.

Question - 51

```
public class A {  
    public A() {  
        System.out.println("A");  
    }  
}  
  
public class Test {  
    public static void main(String... args) throws Exception {
```

```

        A a = new A();

        ObjectOutputStream save = new ObjectOutputStream(new
FileOutputStream("datafile"));
        save.writeObject(a);
        save.flush();

        ObjectInputStream restore = new ObjectInputStream(new
FileInputStream("datafile"));
        A z = (A) restore.readObject();

    }
}

```

What is the output?

- 1.A A
- 2.A
- 3.java.io.NotSerializableException
- 4.None of the above

Explanation :

C is the correct answer.

Class A does not implements Serializable interface. So throws NotSerializableException on trying to Serialize a non Serializable object.

Question - 52

```

public class A implements Serializable{
    public A() {
        System.out.println("A");
    }
}

public class Test {

    public static void main(String... args) throws Exception {
        A a = new A();

        ObjectOutputStream save = new ObjectOutputStream(new
FileOutputStream("datafile"));
        save.writeObject(a);
        save.flush();

        ObjectInputStream restore = new ObjectInputStream(new
FileInputStream("datafile"));
        A z = (A) restore.readObject();
    }
}

```



```
    }  
}  
What is the output?
```

- 1.A A
- 2.A
- 3.Runtime Exception
- 4.Compile with error

Explanation :
B is the correct answer.

On the time of deserialization , the Serializable object not create new object. So constructor of class A does not called.

Question - 53

```
class MyOuter {  
    class MyInner { }  
}
```

after compile how many classes will be created ?

- 1.MyOuter.class and MyOuter\$MyInner.class
- 2.MyOuter\$MyInner.class
- 3.MyOuter.class
- 4.None of the above

Explanation :
A is the correct answer.

two class files will be created. MyOuter.class MyOuter\$MyInner.class

Question - 54

Which statement is more appropriate for Encapsulation ?

- 1.making private is Encapsulation
- 2.Encapsulation refers to the bundling of data with the methods that operate on that data
- 3.both are true
- 4.None of the above

Explanation :
B is the correct answer.

Encapsulation refers to the bundling of data with the methods that operate on that data.

Example

```
public class Position
{
    public double i;
    public double j;

    public int calculatedifference() {
        return i-j;
    }
}
```

This is hiding bussiness logic (how they calculate the difference)

If you want to calculate difference then you have to to

```
Position pt = new Position();
pt.i=20;
pt.j=12;
```

```
pt.calculatedifference();
```

Encapsulation hides the bussiness logic , just send input and get out put.

Question - 55

Fill the gap:

```
public class Position
{
    public double i;
    public double j;

    public int calculatedifference() {
        return i-j;
    }
}

public class Test {

    public static void main(String... args) {

        Position pt = new Position();
        pt.i=20;
        pt.j=12;

        pt.calculatedifference();

    }
}
```

Above example refer to .

Use the following fragments zero or many times

Abstraction

Encapsulation

Composition

Inheritance

Question - 56

Fill the gap:

One object can send message to other like **ClassA** calls **ClassB.getName()** method

is called .

Use the following fragments zero or many times

Abstraction

Dependency

Composition

Inheritance

Question - 57

What is output ?

```
public class Test {  
  
    public static void main(String... args) {  
  
        Pattern p = Pattern.compile("a+b?");  
        Matcher m = p.matcher("aaaabb");  
        boolean b = m.matches();  
        System.out.println(b);  
  
    }  
}
```

- 1.true
- 2.false
- 3.Compile error
- 4.None of the above

Explanation :

B is the correct answer.

a+ means a , one or more times b? means b , once or not at all

Question - 58

What is the output?

```

import java.util.ArrayList;
import java.util.Iterator;
import java.util.List;
import java.util.NavigableSet;
import java.util.TreeSet;

public class Test {
    public static void main(String... args) {

        List<Integer> lst = new ArrayList<Integer>();
        lst.add(34);
        lst.add(6);
        lst.add(2);
        lst.add(8);
        lst.add(7);
        lst.add(10);

        NavigableSet<Integer> nvset = new TreeSet(lst);
        Iterator<Integer> iterator = (Iterator)nvset.iterator();
        while (iterator.hasNext()){
            System.out.print(iterator.next() + " ");
        }

    }
}

```

- 1.Compile error : No method name like iterator()
- 2.34 10 8 7 6 2
- 3.2 6 7 8 10 34
- 4.None of the above

Explanation :

C is the correct answer.

S iterator() Returns the elements in ascending order.

Question - 59

What is the output of the bellow code ?

```

public enum Test {
    BREAKFAST(7, 30), LUNCH(12, 15), DINNER(19, 45);

    private int hh;

    private int mm;

    Test(int hh, int mm) {
        assert (hh >= 0 && hh <= 23) : "Illegal hour.";
        assert (mm >= 0 && mm <= 59) : "Illegal mins.";
        this.hh = hh;
        this.mm = mm;
    }

    public int getHour() {
        return hh;
    }
}

```

```

    public int getMins() {
        return mm;
    }

    public static void main(String args[]){
        Test t = BREAKFAST;
        System.out.println(t.getHour() + ":" + t.getMins());
    }
}

```

1.7:30

2.Compile With Error

3.Compile Without Error but Runtime Exception

4.None of the above

Explanation :

B is the correct answer.

All the enum static like BREAKFAST treat as constructor. Enum constructors Each constant declaration can be followed by an argument list that is passed to the constructor of the enum type having the matching parameter signature. An implicit standard constructor is created if no constructors are provided for the enum type. As an enum cannot be instantiated using the new operator, the constructors cannot be called explicitly. Output is : 7:30

Question - 60

What is the output ?

```

public class Test {
    public static void main(String... args) {
        printUs("ONE", "TWO", "THREE");
        printUs("FOUR", "FIVE");
        printUs(new String[]{"SIX", "SEVEN"});
        printUs();
    }

    private static void printUs(String abc,String... args) {
        System.out.println("Var args method");
        for (String s : args) {
            System.out.println(s);
        }
    }
}

```

1.Var args method ONE TWO THREE Var args method FOUR FIVE Var args method SIX SEVEN Var args method

2.Var args method ONE TWO THREE Var args method FOUR FIVE Var args method SIX SEVEN

3. Var args method SIX SEVEN

4. Compile with error.

Explanation :

D is the correct answer.

```
printUs(new String[]{"SIX", "SEVEN"}); // compile error because atleast one String  
param should be declare. printUs(); // compile error because atleast one String param  
should be declare.
```